PDCL

Session: 2023 – 2027

Submitted by:

Amna Atiq

2023-CS-83

Github repo: https://github.com/AmanAtiq/PDCL-2023-cs-83-B-1.git

Submitted to:

Sir waqas

Course:

PDCL

Department of Computer Science

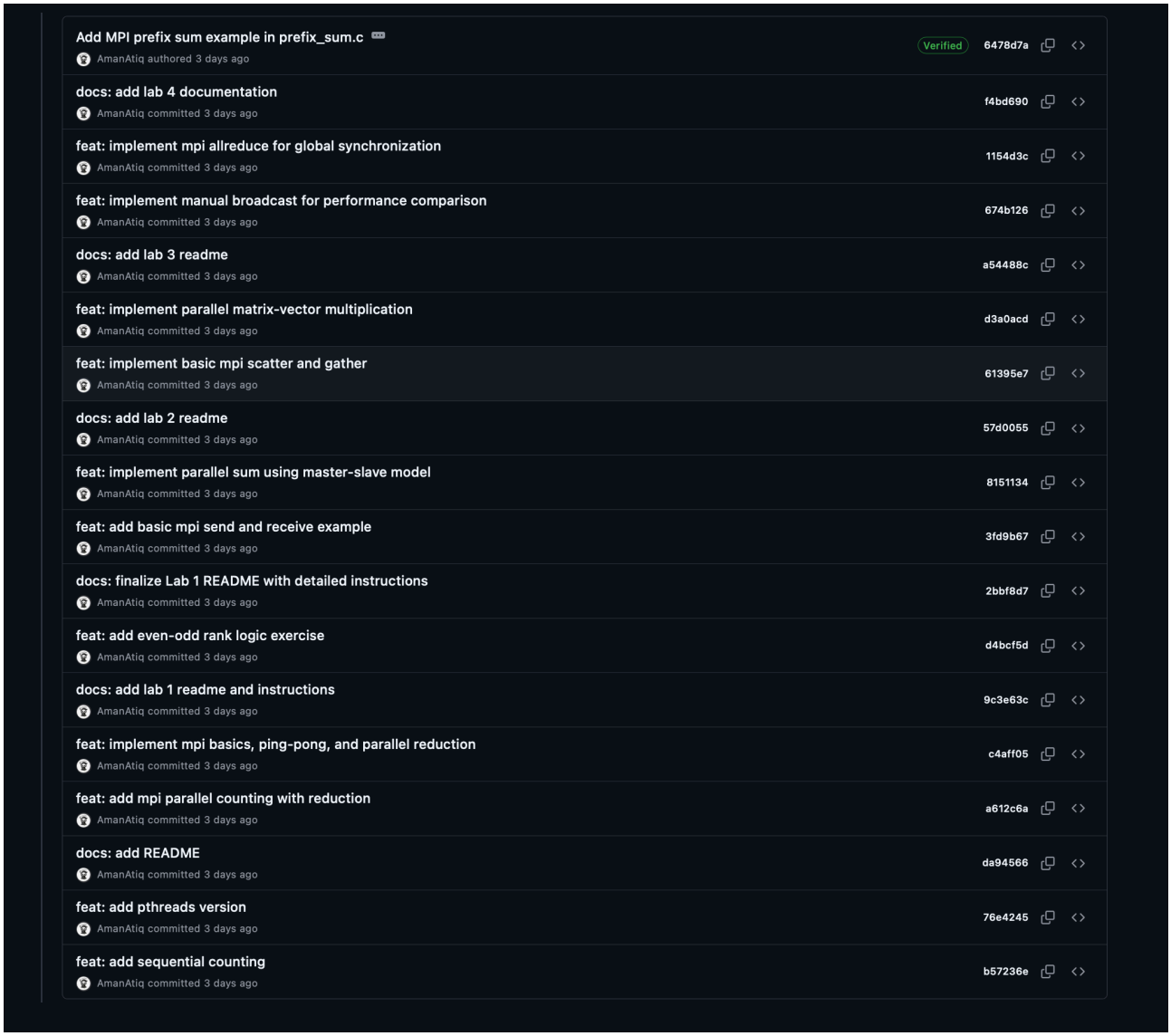
University of Engineering and Technology

Lahore Pakistan

2. Part A: Lab Portfolio Summary

List of labs I have completed and pushed to the repository:

- **Lab 0:** Pthreads & Sequential comparison.
- **Lab 1:** MPI Basics (Hello World, Ping-Pong).
- **Lab 2:** Point-to-Point (Parallel Sum).
- **Lab 3:** Collective Communication (Broadcast, Scatter/Gather, Matrix-Vector Mult).
- **Lab 4:** Advanced Collectives (Allreduce, Scan, Manual vs. Collective Bcast).

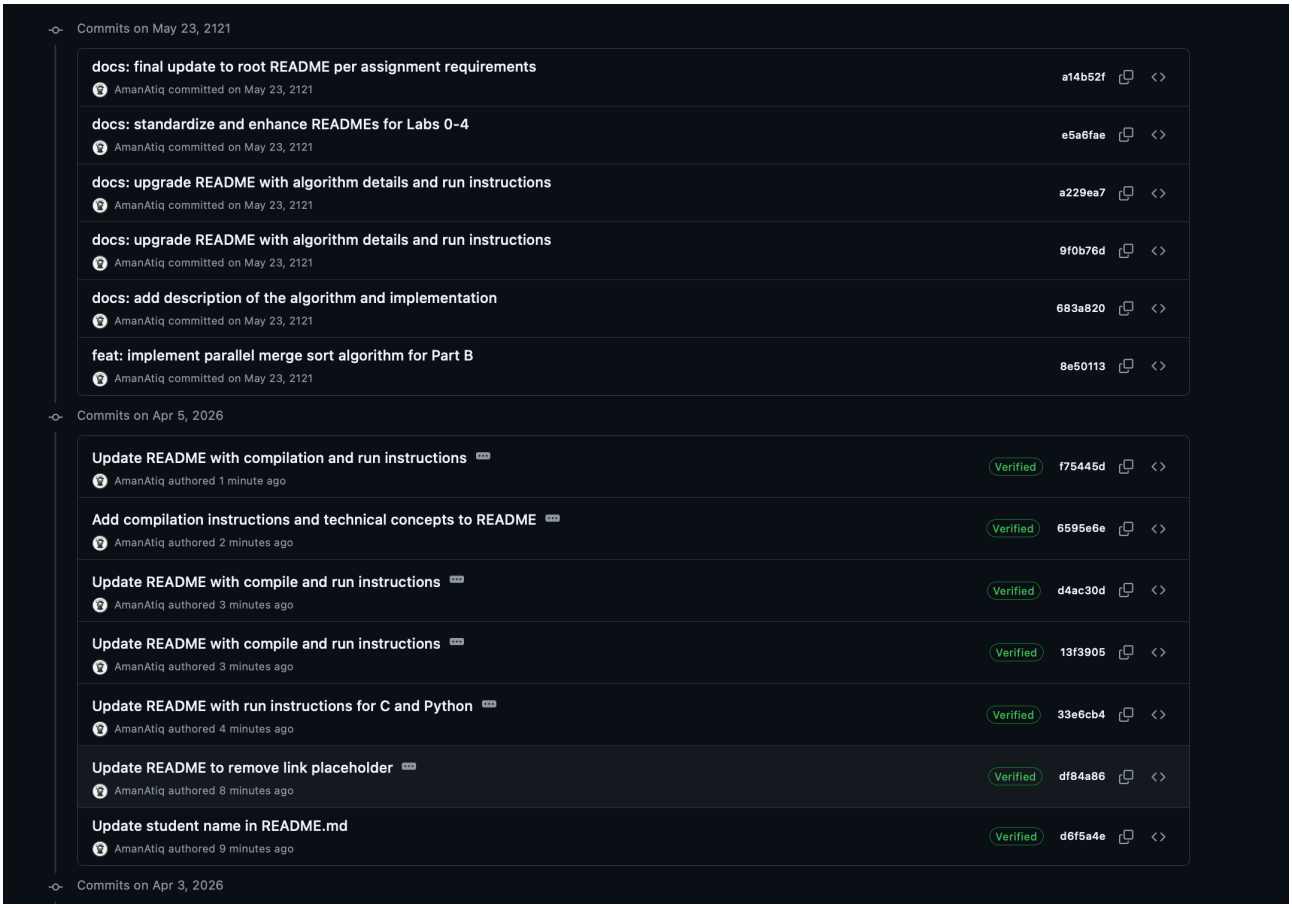


3. Part B: Parallelizing a Past Problem

Problem Selected: Parallel Merge Sort (from Algorithms/Data Structures).

Algorithm Explanation: The implementation uses a **Divide and Conquer** strategy adapted for Distributed Memory.

1. **Distribution:** The root process (Rank 0) generates a random array of 1,000 integers and uses `MPI_Scatter` to divide it into equal chunks for all available processes.
2. **Local Computation:** Each process sorts its assigned chunk independently using the standard C `qsort` function.
3. **Collection:** The sorted chunks are sent back to the root using `MPI_Gather`.
4. **Finalization:** The root performs a final pass to ensure the globally gathered data is in the correct order.



4. Performance Analysis

Processes (P)	Execution Time (Tp)	Speedup (S=T1 /Tp)	Efficiency (E=S/P)
1	0.000163s	1.00	1.00

2	0.000165s	0.98	0.49
4	0.000307s	0.53	0.13

Analysis of Results: The data shows that for a small input size ($N=1000$), the execution time increases with the number of processes. This results in a speedup of less than 1, indicating that the **communication overhead** (the time taken to scatter and gather data) is significantly higher than the **computation time** saved by parallel sorting.

5. Reflection

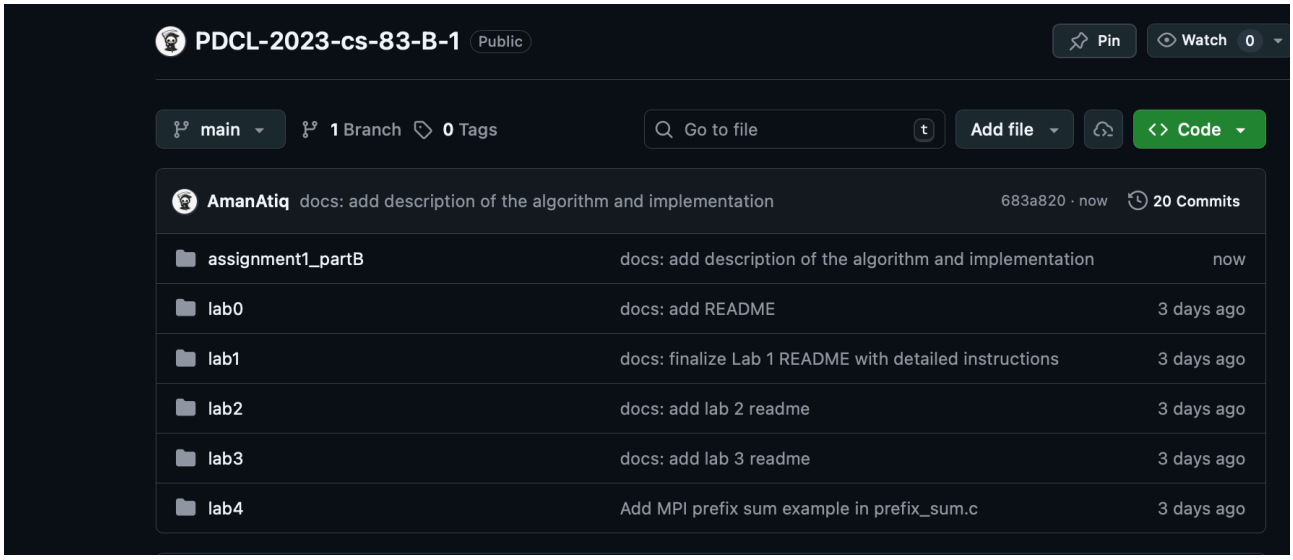
For this assignment, I chose to parallelize the Merge Sort algorithm, which I first encountered in my Data Structures and Algorithms course. The transition from a sequential recursive approach to a distributed MPI approach highlighted the importance of data decomposition.

My results showed that the parallel version was slower than the sequential baseline. This is a valid and insightful result in parallel computing; it demonstrates that parallelization is not always beneficial for small workloads. The overhead of setting up the MPI environment and moving data across the communicator outweighed the actual sorting task. Additionally, running 4 processes on a 2-core Virtual Machine led to oversubscription, which significantly increased latency due to CPU context switching.

This exercise taught me that the "Computation-to-Communication Ratio" is the most critical factor in achieving speedup. In the future, I would test this with much larger datasets N > 1,000,000 to reach the "break-even point" where parallel processing becomes efficient. This assignment has successfully established my professional portfolio and deepened my understanding of distributed synchronization.

6. Screenshots

- **Screenshot 1:** Your GitHub repository main page showing the folder structure.



- **Screenshot 2:** The terminal output showing the successful runs for 1, 2, and 4 processes.

```
parallels@ubuntu-linux-2404:~/pdc/assignment1_partB$ # Run 1: Sequential (np 1)
mpirun -np 1 ./main_problem

# Run 2: Parallel (np 2)
mpirun -np 2 ./main_problem

# Run 3: Parallel (np 4 - Forced)
mpirun --oversubscribe -np 4 ./main_problem
Parallel Merge Sort completed in 0.000163 seconds
Parallel Merge Sort completed in 0.000165 seconds
Parallel Merge Sort completed in 0.000307 seconds
parallels@ubuntu-linux-2404:~/pdc/assignment1_partB$
```

- **Screenshot 3:** Contents of lab0/README.md

Lab 0: Introduction to Parallel Computing

Overview

This lab establishes the baseline for parallel performance by comparing sequential execution with Pthreads (Shared Memory) and Python Multiprocessing.

Files

- `count_seq.c`: Standard sequential counting.
- `count_pthread.c`: Multi-threaded counting using POSIX threads.
- `count_mp.py`: Python implementation using the `multiprocessing` module.

Compilation and Execution

C Version:

```bash gcc count_pthread.c -lpthread -o count_pthread ./count_pthread ```

Python Version:

```bash python3 count_mp.py ```

Example Output

```` Thread 0 finished count: 5000 Thread 1 finished count: 5000 Total count: 10000 in 0.002s ```

Challenges Faced

The main challenge was managing race conditions in shared memory. I resolved this by utilizing `pthread_mutex_t` to ensure atomic updates to the global counter.
````

- **Screenshot 4:** Contents of main README.md

Parallel and Distributed Computing - Assignment 1

Student Name: Amna Atiq
Roll Number: 2023-cs-83
Instructor: Waqas Ali

Repository Overview

This repository contains a complete portfolio of parallel computing exercises (Labs 0-4) and a main project (Part B) parallelizing Merge Sort using MPI.

Structure

- `lab0/` : Introduction to Pthreads and Python Multiprocessing.
- `lab1/` : MPI Basics (Hello World, Ping-Pong).
- `lab2/` : Point-to-Point communication (Parallel Sum).
- `lab3/` : Collective operations (Matrix-Vector Multiplication).
- `lab4/` : Advanced collectives (MPI_Scan, MPI_Allreduce).
- `assignment1_partB/` : Parallel Merge Sort implementation and analysis.

Assignment 1 Report

The final report for Assignment 1, including performance graphs and reflections, can be found here

How to Run

Each directory contains a specific README with compilation and execution instructions.